

INITIAL PROJECT INVESTIGATION & PLAN

Accelerating a Convolutional Neural Network on the Kria KV260 FPGA

Custom Vitis HLS accelerator cores on Zynq UltraScale+ programmable logic

Gilbert Tong

Shuruthy Dhushiyandan

Kaira Dumasia

1 Problem Outline

Compute cost

CNN inference is made up of billions of multiply-accumulate (MAC) operations with lots of deeply nested loops which is expensive on a general purpose CPU.

Edge constraints

Embedded and real-time vision needs high throughput and energy efficiency that a normal processor struggles to deliver.

FPGA opportunity

Zynq devices pair an ARM CPU (PS) with programmable logic (PL) which can build custom parallel datapaths to do many MACs per clock.

Our task

Implement FPGA Acceleration of CNN Convolution on the Kria KV260 and Performance Evaluation against Software Execution on the Processing System

Why it matters

GPUs already accelerate CNNs by executing operations in parallel.

On Zynq, that same parallelism is achievable in the programmable logic, which allows us to maintain low power and keep it fully under our control.

Success = speedup

We will measure the hardware accelerated design's latency and throughput and compare it against a software only baseline running on the ARM cores

2 Project Specific Aims

1 Develop a software baseline implementation

Implement a single 3×3 convolution layer in C/C++ running entirely on the ARM Cortex-A53 processor of the Kria KV260. Verify the correctness of the output and establish baseline performance metrics such as execution time and throughput.

- ❑ Working software implementation
- ❑ Correct convolution outputs
- ❑ Baseline timing measurements

3 Optimise the FPGA Accelerator

Improve the performance of the hardware implementation using FPGA optimisation techniques such as pipelining, loop unrolling, array partitioning, and efficient memory access. Evaluate the impact of each optimisation on latency and FPGA resource utilisation.

- ❑ Optimised HLS design
- ❑ Resource utilisation reports (LUTs, DSPs, BRAMs)
- ❑ Timing improvements after each optimisation

2 Design and Implement an FPGA Convolution Accelerator

Develop a hardware accelerator for the same convolution layer using Vitis HLS. The accelerator will implement the convolution operation in programmable logic (PL) and communicate with the processing system through AXI interfaces.

- ❑ HLS convolution kernel
- ❑ FPGA bitstream
- ❑ Successful PS–PL communication
- ❑ Correct hardware results matching the software implementation

4 Evaluate Hardware vs Software Performance

Compare the FPGA accelerator against the software implementation using quantitative performance metrics. Analyse execution time, throughput, speedup, and FPGA resource usage to determine the effectiveness of hardware acceleration for CNN convolution.

- ❑ CPU vs FPGA benchmarks
- ❑ Speedup calculations
- ❑ Performance graphs and tables
- ❑ Discussion of trade-offs between performance and hardware resource usage

3 Algorithm to be Accelerated

Target model: LeNet-5 on MNIST

A CNN classifies an image by sliding small learned filters over it (convolution), downsampling, then fully connected layers produce 10 class scores.

Convolution is a six deep loop nest of MACs which is the dominant cost and out acceleration target.

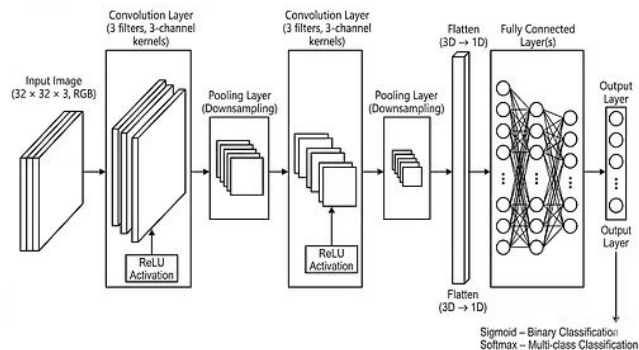
LeNet-5: ~62k parameters, ~417k MACs/image. Small enough that all weights + activations fit in on-chip BRAM, so the design is compute-bound.

The MNIST (Modified National Institute of Standards and Technology) dataset is a massive collection of 70,000 grayscale, 28×28 pixel images of handwritten digits (0–9).

Where the parallelism lives

- Across output channels (independent filters)
- Per-pixel reduction (all taps multiply at once)
- Across output pixels (line buffers + sliding window)
- Layer pipelining across the network (HLS DATAFLOW)

Compute is dominated by convolution



LeNet-5 architecture

4 Investigation So Far

● Studied the CNN algorithm

Reviewed how CNN inference works: convolution, pooling, and fully connected layers and identified the six nested MAC loops in the convolution as the dominant computation cost

● Chose model and dataset

Selected LeNet-5 on MNIST, because it is small enough to prototype quickly (around 62k parameters), while still representing the convolution heavy pattern we are accelerating

● Defined the accelerator approach

Decided on a layered build: accelerate a single convolution first to validate the HLS to hardware flow, then potentially extend to the remaining layers

● Profiled the workload

Estimated per-layer computation cost, confirmed convolution accounts for the large majority of MACs, with all weights and activations small enough to fit in on-chip BRAM.

● Surveyed the KV260 hardware

Studied the Zynq UltraScale+ architecture: ARM Cortex-A53 cores (PS) paired with programmable logic (PL), and confirmed Vitis HLS as the design flow

● Reviewed reference designs

AMD/Xilinx. *Vitis In-Depth Tutorials (HLS, Hardware Acceleration, CNN workflows)*

<https://github.com/Xilinx/Vitis-Tutorials>

AMD/Xilinx. *Vitis AI / CNN Acceleration Framework (official machine learning acceleration stack)*

<https://github.com/Xilinx/Vitis-AI>

Xilinx. *LeNet AI Engine / CNN System-Level Tutorial (PS + PL + AI Engine design flow)*

<https://xilinx.github.io/Vitis-Tutorials/>

sherylll. *LeNet-5 FPGA Accelerator (HLS-based CNN implementation)*

<https://github.com/sherylll/lenet5-accelerator>

changwoolee. *LeNet-5 HLS CNN Accelerator (Vitis/Vivado HLS convolution design)*

https://github.com/changwoolee/lenet5_hls

5

Project Plan: Milestones & Timeline

Weeks	Milestone	Owner	Deliverable
Week 6	Train LeNet-5 baseline, quantise weights, produce golden test vectors	Kaira	Float32 baseline, fixed-point format, golden vectors
Week 6-7	Build ARM-only software baseline on the KV260	Kaira and Gilbert	SW-only latency/throughput baseline
Week 7	Implement unoptimised Conv1 HLS C++, validate against golden vectors	Shuruthy	Functionally-correct Conv1 (C-sim pass)
Week 8	Optimise Conv1 (pragmas added incrementally), track ablation table, C/RTL sim	Shuruthy	Optimised Conv1 HLS + ablation results
Week 8-9	Integrate Conv1 IP into Vivado block design and PS, bring up on board	Gilbert	Conv1 accelerator running on KV260
Week 9	Benchmark Conv1 hardware vs. software baseline	Kaira and Gilbert	Conv1 speedup + accuracy results
Week 10	Extend to remaining layers if time allows, finalise report and demo	All	Final results + report/demo

6

Risks & Contingencies

Timing closure / resource overuse

HIGH

Risk: FPGA design exceeds timing or uses too many LUTs/DSPs

Impact: Design fails to meet target frequency or cannot be implemented

Prevention: Incremental optimisation (pipeline → unroll), monitor HLS reports, start simple

Quantisation accuracy loss

MEDIUM

Risk: Fixed-point representation reduces numerical precision

Impact: Output mismatch between CPU and FPGA results

Prevention: Start with floating-point, then gradually convert to fixed-point, validate at each step

Model being too large

MEDIUM

Risk: CNN design too large for FPGA resources

Impact: Cannot synthesise or long compile times

Prevention: Keep model small (1 layer, 4–8 filters), reduce resolution, avoid full CNN

Data movement negates speedup

HIGH

Risk: AXI/DMA transfer time dominates computation time

Impact: FPGA shows little or no real performance gain

Prevention: Use streaming/BRAM, minimise transfers, batch data movement, measure compute separately

Toolchain / version friction

MEDIUM

Risk: Vitis/Vivado version issues or setup incompatibility

Impact: Build failures or inability to deploy to KV260

Prevention: Use recommended stable toolchain, test early with simple HLS kernel

Board access / hardware time

LOW

Risk: Limited access to single KV260 board

Impact: Delays in testing and integration

Prevention: Parallelise work, rely on simulation early, schedule board usage

Roles of Team Members

Gilbert Tong

Hardware/Software Integration

- Building the Vivado block design and integrating with the Zynq PS
- Generating the bitstream and setting up the Petalinux overlay
- Writes the driver control layer on the ARM side in C++ to load images into the accelerator, triggers it and sends back the results.
- Builds the ARM only software baseline

Shuruthy Dhushiyandan

Hardware Developer

- Implements each layer as HLS C++, starting from a functionally-correct unoptimized version validated against Kaira's golden vectors.
- Adds the pragmas incrementally and re-run C sim to check for correctness
- Track the ablation table
- Runs the C/RTL co-sim and export the IP block over to Gilbert

Kaira Dumasia

Software Developer/AI trainer

- Train LeNet-5 on MNIST in Python and get a solid float32 baseline accuracy.
- Simulate various bit widths to keep the accuracy within 1%.
- Extract and format the trained weights into whatever HLS layer we tend to use.
- Produce the golden output test vectors to be used on the C sim and the board.
- Build accuracy and latency benchmarking scripts